Universidad Nacional Autónoma de México

Facultad de Ciencias

Licenciatura en Ciencias de la computación

Sistemas Operativos

Desarrollo de un monitor básico del sistema

Alumnos:
Ledesma Cuevas Eduardo
Méndez Camacho Monserrat

Profesor:
Javier León Cotonieto

Martes 26 de mayo de 2026

Índice general

1. Introducción **3**

2. Marco Teórico **4**
2.1. Monitores de sistema en Linux 4
2.2. El directorio /proc 4
2.3. Procesos 7
2.4. Ncurses 9

3. Desarrollo **10**
3.1. Organización del proyecto 10
3.2. Módulos implementados 10
3.3. Flujo de ejecución 12

4. Resultados **13**

5. Conclusiones **14**

Introducción

El proyecto final desarrollado para la materia de Sistemas Operativos consiste en la implementación de un Monitor Básico del Sistema para Linux. Este monitor permite a los usuarios supervisar en tiempo real los procesos en ejecución y el uso de los principales recursos del sistema, como la memoria y la CPU. Además, incluye secciones que muestran información sobre el estado del disco y la actividad de la red.

El objetivo principal de contar con un monitor de sistema es proporcionar una herramienta que facilite la observación del comportamiento de los procesos y su impacto en los recursos disponibles. De esta manera, los usuarios pueden identificar posibles problemas de rendimiento, detectar cuellos de botella y evaluar el consumo de recursos por parte de aplicaciones y servicios.

Existen programas ampliamente utilizados con este mismo propósito, como `top`, que viene instalado de forma predeterminada en la mayoría de las distribuciones Linux, o `htop`, una versión mas avanzada que también se ejecuta en la línea de comandos y ofrece información adicional así como la capacidad para que el usuario puede manipular los procesos. Gracias a estas herramientas, es posible obtener un desglose completo de los procesos activos, uso de recursos y una visualización completa de los componentes principales de nuestro equipo.

El monitor desarrollado en este proyecto se inspira en estas herramientas, pero se centra en mostrar de manera organizada y clara la información más relevante para el usuario dentro de la terminal. Se hace un desglose de los procesos con su respectivo ID, user, uso de CPU, memoria, así como otros datos relevantes. Con ello, se busca no solo replicar un sistema ya existente, sino también comprender cómo el sistema operativo expone datos internos a través del directorio `/proc`, y cómo estos son interpretados.

Marco Teórico

2.1. Monitores de sistema en Linux

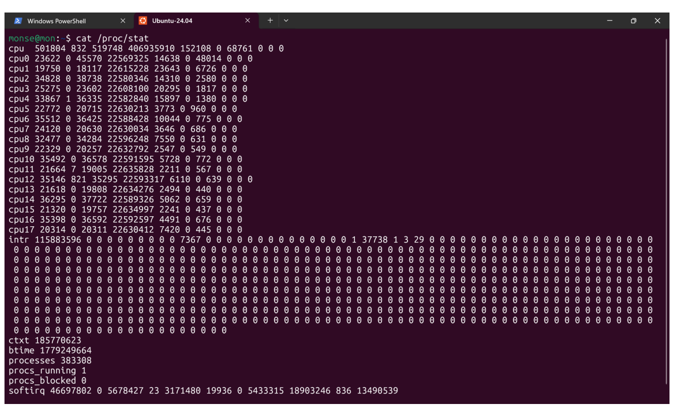
Los monitores de sistema son herramientas fundamentales que permiten observar en tiempo real los procesos en ejecución y el rendimiento general de un equipo. Estos programas ofrecen métricas sobre el uso de CPU, memoria, disco, red, etc., facilitando la administración y el diagnóstico de problemas. Uno de los monitores más conocidos es `top`, que viene integrado de forma predeterminada en la mayoría de distribuciones Linux. Este monitor proporciona una visión dinámica del estado del sistema, mostrando procesos activos y estadísticas de recursos.

2.2. El directorio /proc

En Linux, todo se gestiona como si fuera un archivo. El directorio `/proc` es un pseudo-filesystem que actúa como interfaz entre el usuario y las estructuras internas del kernel.
A diferencia de los archivos tradicionales, los contenidos de `/proc` son archivos virtuales. Esto son archivos que no existen físicamente en el disco, sino que son generados por el kernel en el momento en que se leen. Este directorio se crea cada vez que el sistema arranca y refleja el estado actual del sistema operativo.
La mayoría de los archivos en `/proc` son de solo lectura, aunque algunos, como los ubicados en `/proc/sys`, permiten escritura para modificar parámetros del kernel en tiempo real. Por ello `/proc` es una herramienta esencial para monitorear y ajustar el comportamiento del sistema.

/proc/stat

El archivo `/proc/stat` contiene información agregada sobre la actividad del kernel desde el último arranque.



La primera línea, identificada como cpu, resume el tiempo de uso de todas las CPUs, mientras que las líneas de cada cpuN detallan cada núcleo individual. Los valores representan tiempos acumulados en diferentes estados de ejecución:

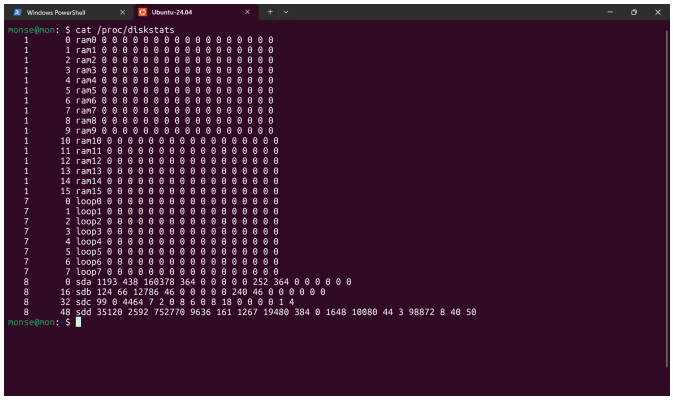
- `user`: tiempo en procesos de usuario.
- `nice`: procesos en modo usuario con prioridad ajustada.
- `system`: tiempo en procesos del kernel.
- `idle`: tiempo en reposo.
- `iowait`: tiempo de espera de operaciones de entrada/salida.
- `irq / softirq`: atención a interrupciones de hardware y software.
- `steal`: tiempo en espera involuntaria.
- `guest / guest_nice`: tiempo ejecutando máquinas virtuales invitadas.

Estos datos permiten calcular métricas como el porcentaje de uso de CPU y detectar cuellos de botella.

/proc/diskstats

El archivo `/proc/diskstats` expone estadísticas detalladas de operaciones de entrada y salida a nivel de bloque para cada dispositivo de almacenamiento. Cada línea corresponde a un dispositivo y contiene campos como:

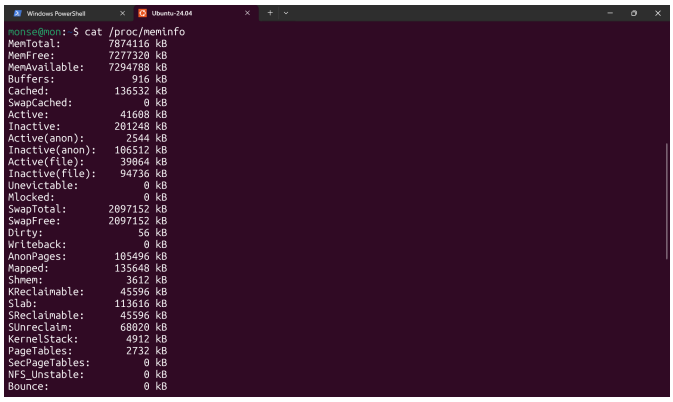
- Lecturas y escrituras completadas.
- Sectores leídos y escritos.
- Tiempo total invertido de operaciones de E/S.
- Número de operaciones en curso.



A partir del kernel 4.18 se incluyen métricas adicionales relacionadas con operaciones de descarte (discard), útiles en dispositivos SSD. Las estadísticas presentadas en esta entrada permiten analizar el rendimiento del almacenamiento secundario y detectar problemas con latencia.

/proc/meminfo

el archivo `/proc/meminfo` proporciona un desglose detallado sobre la distribución y utilización de memoria. Cuando se desea conocer estadísticas como memoria usada y disponible, espacio de intercambio o caché y buffers, podemos analizar el contenido de este archivo.



Entre los campos más relevantes se encuentran:

- `MemTotal:` cantidad total de RAM utilizable.
- `MemFree:` RAM libre, memoria que no se utiliza para nada en absoluto
- `MemAvailable`: memoria disponible para nuevos procesos, considerando cachés y buffers.

Así como información sobre el espacio de intercambio (swap) del sistema. Este es un espacio de respaldo en el disco para la RAM. Cuando la memoria física esta llena el sistema utiliza el espacio de intercambio:

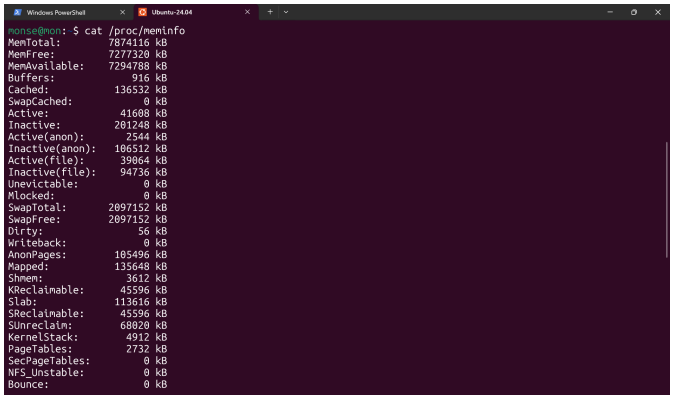
- `SwapTotal:` tamaño total del espacio de intercambio de disco disponible.
- `SwapFree:` espacio de intercambio no utilizado.

Analizar estos valores permite conocer el estado de la memoria física y del área de intercambio, los cuales son esenciales en un monitor del sistema.

/proc/net/dev

Dentro del directorio `/proc/net`, el archivo `/proc/net/dev` contiene estadísticas de los dispositivos de red. Estas estadísticas son esenciales para evaluar el tráfico de red y detectar fallos en la comunicación.
Entre los datos disponibles se incluyen:

- Paquetes recibidos y enviados.
- Bytes transferidos.
- Número de errores y colisiones



/proc/uptime

El archivo `/proc/uptime` contiene dos valores:

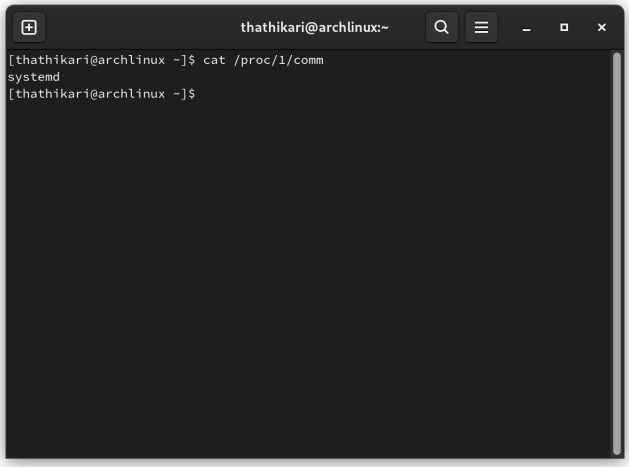
1. los segundos que el sistema ha estado en ejecución desde el arranque y
2. el tiempo acumulado en que la CPU ha permanecido en estado inactivo o reposo.

2.3. Procesos

Cada proceso en Linux se presenta mediante un subdirectorio dentro de `/proc`, identificado por su PID (ID del proceso).
Estos directorios contienen información detallada sobre el estado del proceso, sus recursos consumidos, archivos abiertos, etc. El acceso a esta información es clave para construir un monitor de sistema que muestre actividad de procesos en tiempo real.

/proc/pid/comm

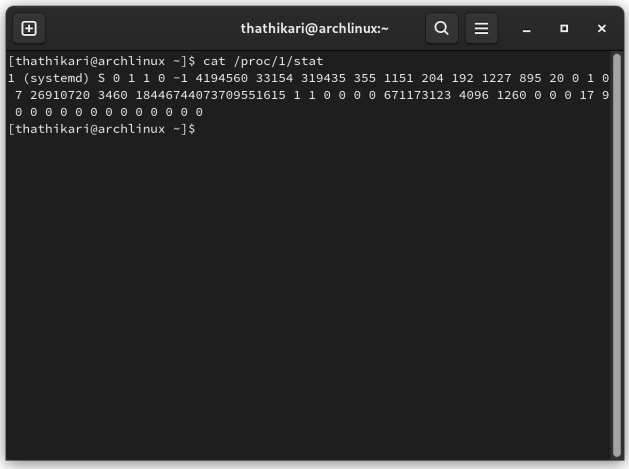
El archivo `/proc/<pid>/comm` contiene únicamente un valor, correspondiente al nombre del proceso al que pertenece el `pid` consultado.



/proc/pid/stat

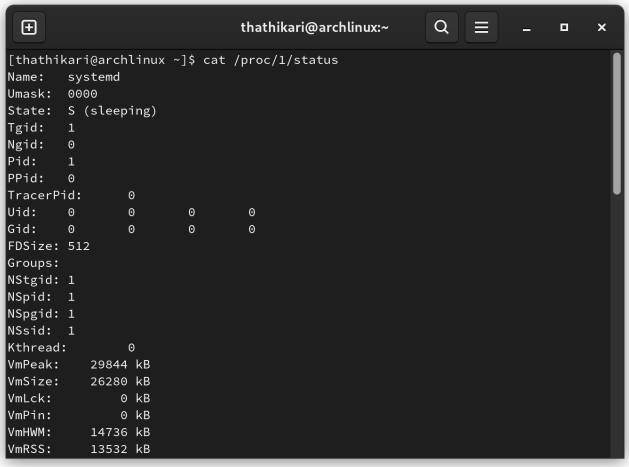
El archivo `/proc/<pid>/stat` contiene información de estado del proceso repartidos en un total de 52 campos. Entre los datos presentes se encuentran:

- Estado del proceso
- Tiempo de CPU en modo usuario
- Tiempo de CPU en modo kernel
- Memoria virtual del proceso
- Resident Set Size



/proc/pid/status

El archivo `/proc/<pid>/status` contiene mucha de la información presente en `/proc/<pid>/stat` con la diferencia de estar presentada en un formato más amigable para los humanos.



2.4. Ncurses

Ncurses es una biblioteca que permite desarrollar interfaces de usuario basados en texto TUI (Text User Interfaces) dentro de la terminal. Esta biblioteca proporciona herramientas para crear ventanas, manejar colores y organizar la pantalla de manera dinámica.
Ncurses significa new curses, ya que es un reemplazo de curses que se encuentra descontinuado.

Desarrollo

El desarrollo del monitor se llevó a cabo de manera modular y secuencial, con el objetivo de obtener una aplicación capaz de mostrar estadísticas verídicas sobre el uso de los recursos de un sistema Linux. Para ello, se realizó una recopilación de los archivos del directorio `/proc` necesarios. También se diseño una arquitectura modular donde se divide el software en módulos más pequeños e independientes. Cada módulo cumple una función específica y puede operar de manera independiente o en conjunto con otros, formando así un sistema completo.
El objetivo del diseño modular del proyecto es para:

- Mantenimiento más sencillo: Al estar dividido en secciones independientes, cualquier problema puede ser localizado y resuelto sin afectar a otras partes del sistema.

- Actualizaciones rápidas y seguras: se pueden agregar nuevas funcionalidades a un módulo específico sin alterar el resto del software, acelerando los procesos de actualización.

La aplicación se implementó en lenguaje C y se apoyó en la biblioteca de `ncurses`, lo que permitió construir una interfaz basada en texto (TUI) dentro de la terminal.

3.1. Organización del proyecto

El proyecto se estructuró en dos directorios principales:

- `include/`: Este directorio contiene los archivos de cabecera (.h) que definen las funciones y estructuras de cada módulo.

- `src/`: El siguiente archivo contiene los archivos fuente (.c) donde se implementa la lógica de cada módulo.

Además, se incluyó un `Makefile` para automatizar la compilación y un archivo `README.md` con la documentación básica necesaria así como los pasos para la ejecución del proyecto. Como se menciono anteriormente, esta organización favorece la claridad y el mantenimiento del código, ya que cada recurso del sistema (cpu, mem, etc) se encuentra encapsulado en su propio módulo.

3.2. Módulos implementados

Cada módulo corresponde a un recurso del sistema y se relaciona directamente con un archivo del directorio `/proc`. A continuación se describen los principales:

CPU

Archivos `cpu.c` y `cpu.h` Para obtener estadísticas correspondientes al CPU se utilizo el archivo `/proc/stat`. En dicho archivo se lee los tiempos acumulados de la CPU en diferentes estados como user, system, idle, iowait, etc.
En el código se calcula el porcentaje total de CPU que se esta utilizando en tiempo real. La interfaz muestra el total del sistema.

Memoria

Archivos `memory.c` y `memory.h` Para la sección de memoria se utilizo el archivo de `/proc/meminfo`. Aquí se obtienen estadísticas de memoria RAM y espacio de intercambio (swap). Los datos relevantes para el desarrollo del monitor son:

- MemTotal
- MemFree
- MemAvailable
- SwapTotal
- SwapFree

En la interfaz se presenta al usuario la memoria total, libre y disponible, así como el estado del área de intercambio.

Disco

Archivos `disk.c` y `disk.h` En el archivo `/proc/diskstats` se encuentra información correspondiente a las operaciones de lectura y escritura en dispositivos de almacenamiento. Para este módulo nos interesamos por los datos correspondientes a los sectores de escritura y lectura procesados. Es importante notar que independientemente de la configuración de las particiones, Linux reporta estas estadísticas en sectores de 512 bytes.

Red

Archivos `network.c` y `network.h` Para la sección de red de utilizo el archivo `/proc/net/dev`, que tiene la funcionalidad de recopilar información sobre paquetes enviados y recibidos, bytes transferidos y errores de comunicación.
La salida en la interfaz presenta al usuario un resumen del tráfico de red, con la cantidad de bytes enviados y recibidos al momento, así como un acumulado del total.

Tiempo de actividad

Archivos `uptime.c` y `uptime.h` En el archivo `/proc/uptime` se nos proporciona el tiempo total que el sistema ha estado en ejecución y el tiempo en que la CPU ha permanecido inactiva. En este caso, deseamos mostrar en la interfaz el tiempo de actividad del sistema (en segundos).

Procesos

Archivos `process.c` y `process.h` Para la lectura de procesos, se hace uso de múlitples archivos en `/proc`. Para obtener la información de cada proceso, se itera por todos los archivos `/proc/<pid>` correspondientes a cada uno de los procesos en ejecución. Utilizando `/proc/<pid>/comm`, `/proc/<pid>/stat` y `/proc/<pid>/status` se obtiene la información relevante de cada proceso, como uso de CPU y memoria, usuario dueño del proceso, tiempo de ejecución y estado del proceso. En el código se hace uso de las *syscalls* `sysconf(_SC_PHYS_PAGES)` y `sysconf(_SC_PAGESIZE)` para obtener el número de páginas en RAM y el tamaño de las mismas, de esta forma obtenemos la RAM total de forma más directa que con `/proc`. De forma similar, para obtener el Resident Set Size (RSS) en KB de los procesos, es necesario conocer el tamaño de las páginas, pues `/proc` lo reporta como páginas en lugar de bytes.

Interfaz

Archivos `ui.c` y `ui.h` Para la interfaz se usó la biblioteca `ncurses`, que nos permite organizar la información en ventanas y refrescar la pantalla periódicamente para mostrar la información en tiempo real.

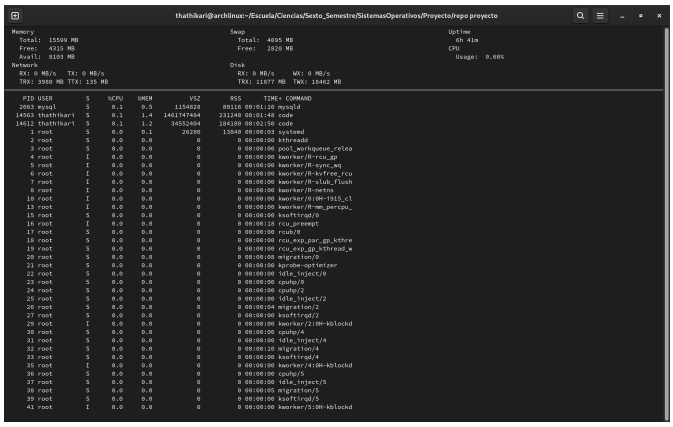
3.3. Flujo de ejecución

El programa inicia en `main.c`, donde se inicializa la interfaz `ncurses` y se llaman los módulos correspondientes. El flujo es el siguiente:

1. Inicialización de la interfaz.
2. Lectura de archivos `/proc` por cada módulo
3. Procesamiento de datos y cálculo de métricas.
4. Refresh constante.
5. Finalización del programa cuando el usuario lo ordene.

Resultados

A continuación se muestran capturas del monitor de recursos en funcionamiento corriendo en un sistema Linux.



Se aprecian todas las métricas de memoria, swap, uptime, uso de red y disco así como el uso del CPU y la lista de procesos corriendo junto con su información correspondiente.
Se adjunta un link con un video mostrando el funcionamiento y actualización en tiempo real de los recursos en el monitor.

Video demostración

Conclusiones

El monitor básico del sistema desarrollado como parte del proyecto final del curso de Sistemas Operativos constituye una herramienta útil y accesible para supervisar los procesos y el uso de recursos en un sistema Linux. Su diseño modular y el empleo de la biblioteca `ncurses` permiten ofrecer una interfaz intuitiva y organizada dentro de la terminal, facilitando la visualización en tiempo real de métricas clave como el consumo de CPU, memoria, disco y red.

Este proyecto proporciona una base practica para comprender cómo el sistema operativo expone información interna a través del directorio `/proc`, y cómo esta puede ser interpretada y presentada de manera clara y eficiente.